

Algoritmi e Strutture Dati – 14/7/2025

Esercizio A1 – Complessità – Punti ≥ 8

È possibile osservare che $T(n) = \Omega(n^2)$, a causa del termine n^2 che compare nell'equazione di ricorrenza. Verifichiamo ora se vale anche $T(n) = O(n^2)$, utilizzando il metodo di sostituzione (per tentativi). In tal caso, avremmo $T(n) = \Theta(n^2)$.

Caso base Nel caso $n = 1$, si ottiene

$$T(1) = 1 \leq c \cdot 1^2$$

che è verificata per $c \geq 1$. Attenzione però: nel caso $n = 2$, la divisione per $\sqrt{5}$ dà origine a $\lfloor 2/\sqrt{5} \rfloor = 0$, ma $n = 0$ non può essere verificato come caso base:

$$T(0) = 1 \leq c \cdot 0^2$$

che è falsa.

Dobbiamo quindi verificare il caso $n = 2$:

$$\begin{aligned} T(2) &= T\left(\left\lfloor \frac{2}{\sqrt{2}} \right\rfloor\right) + 2T\left(\left\lfloor \frac{2}{\sqrt{5}} \right\rfloor\right) + 2^2 \\ &= T(1) + 2T(0) + 4 \\ &= 1 + 2 + 4 = 7 \leq c \cdot 2^2 \end{aligned}$$

che è verificata per $c \geq 7/4$.

Passo induttivo Supponiamo che esistano costanti $c > 0$ e $m \geq 1$ tali che, per ogni $k < n$, valga l'ipotesi induttiva:

$$T(k) \leq ck^2$$

Verifichiamo se l'ipotesi regge nel passo induttivo:

$$\begin{aligned} T(n) &= T\left(\left\lfloor \frac{n}{\sqrt{2}} \right\rfloor\right) + 2T\left(\left\lfloor \frac{n}{\sqrt{5}} \right\rfloor\right) + n^2 \\ &\leq T\left(\frac{n}{\sqrt{2}}\right) + 2T\left(\frac{n}{\sqrt{5}}\right) + n^2 \quad (\text{trascuriamo l'intero inferiore}) \\ &\leq c\left(\frac{n}{\sqrt{2}}\right)^2 + 2c\left(\frac{n}{\sqrt{5}}\right)^2 + n^2 \\ &= c \cdot \frac{n^2}{2} + 2c \cdot \frac{n^2}{5} + n^2 = \left(\frac{9}{10}c + 1\right)n^2 \end{aligned}$$

Per avere $T(n) \leq cn^2$, è sufficiente che:

$$\frac{9}{10}c + 1 \leq c \quad \Rightarrow \quad c \geq 10$$

Abbiamo quindi trovato $m = 1$, $c = \max(1, 7/4, 10) = 10$ e quindi $T(n) = O(n^2)$. Poiché $T(n) = \Omega(n^2)$, possiamo concludere che $T(n) = \Theta(n^2)$.

Esercizio A2 – Albero bello – Punti ≥ 10

Questo problema è simile a quello di identificare il più grande albero perfetto contenuto in un albero, proposto nel luglio 2023. La funzione principale richiama una funzione ricorsiva che prende in input un nodo t e restituisce due valori: la dimensione del più grande alberobello contenuto nel sottoalbero radicato in t e la dimensione dell'alberobello radicato in t , se esiste, o -1 altrimenti.

Per costruire la funzione ricorsiva, osserviamo che dato un nodo t :

- Se t è **nil**, non è un alberobello;
- Se t è una foglia, è un alberobello di dimensione 1;

- Se uno dei figli di t è **nil** oppure il sottoalbero radicato in esso non è un alberobello, t non è un alberobello; in questo caso, restituiamo come dimensione massima il massimo tra i valori trovati nei due sottoalberi;
- Se entrambi i sottoalberi radicati nei figli di t sono alberibelli, anche t è un alberobello. La sua dimensione è pari alla somma delle dimensioni dei due sottoalberi più uno (il nodo stesso). In questo caso, questa dimensione coincide sia con quella dell'alberobello radicato in t , sia con quella massima trovata finora.

La complessità computazionale è ovviamente $O(n)$, in quanto si effettua una post-visita dell'albero.

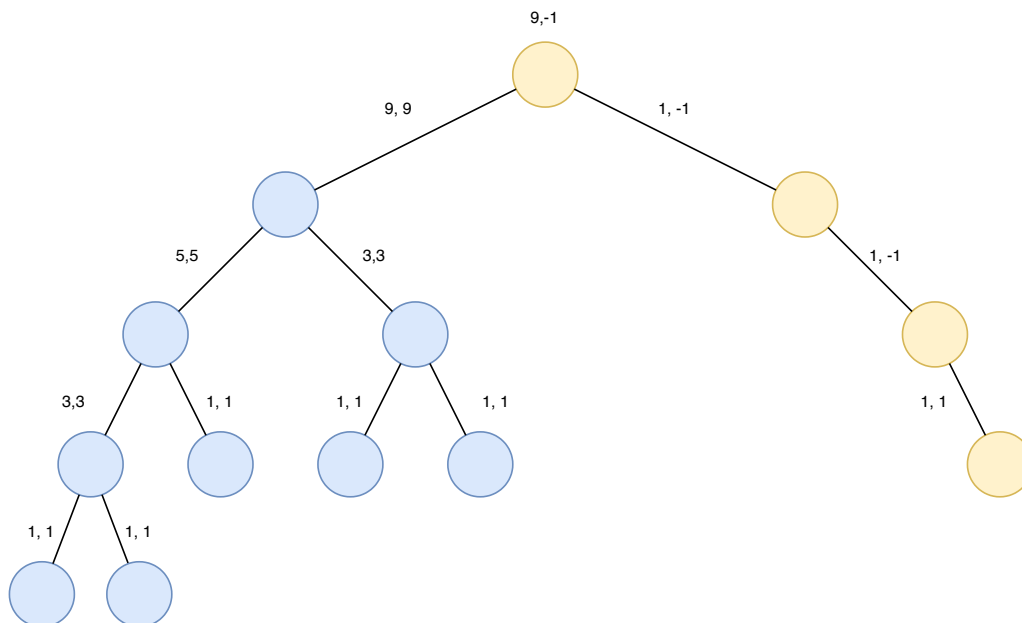
```
int largestAB(TREE T)
```

```
    int maxSoFar, curr = abRec(T)
    return maxSoFar
```

```
int, int abRec(TREE t)
```

```
    if t == nil then
        | return 0, -1
    else if t.left == t.right == nil then
        | return 1, 1
    int, int maxSoFarL, currL = abRec(t.left)
    int, int maxSoFarR, currR = abRec(t.right)
    if currL == -1 or currR == -1 then
        | return (max(maxSoFarL, maxSoFarR), -1)
    int curr = currL + currR + 1
    return (curr, curr)
```

La figura seguente mostra i valori ritornati nell'albero di esempio.



Esercizio A3 – Most distant – Punti ≥ 12

La soluzione proposta si ispira al problema della *Distanza fra partizioni* visto nelle esercitazioni, ed esegue una visita in ampiezza (BFS) simultaneamente da tutte le celle contenenti terraferma. Si inizializza una matrice delle distanze in cui tutte le celle di terra partono con distanza 0, mentre le celle di mare sono inizialmente impostate a $+\infty$, per indicare che non sono ancora state raggiunte. Tutte le celle di terra vengono inserite in una coda. Durante la BFS, per ogni cella estratta dalla coda si esaminano le celle adiacenti. Se una cella adiacente non è ancora stata visitata, le si assegna una distanza pari a quella della cella corrente più uno, e la si inserisce in coda. In questo modo, si propaga gradualmente la distanza a partire dalle zone di terra.

Al termine della visita, si restituisce il valore massimo della matrice. Tale valore rappresenta la distanza, misurata in termini di distanza di Manhattan, della cella di mare più lontana da qualsiasi tratto di terra.

La complessità dell'algoritmo è $\Theta(n^2)$, dominata dall'inizializzazione della matrice delle distanze e dalla visita di ciascuna cella al più una volta.

```
mostDistant(boolean[][] A, int n)
```

```
    QUEUE Q = Queue()
    int[][] dist = new int[1..n][1..n] = {∞}
    for r = 1 to n do
        for c = 1 to n do
            if A[r][c] then
                Q.enqueue((r, c))
                dist[r][c] = 0
    while not Q.isEmpty() do
        int, int r, c = Q.dequeue()
        foreach dr, dc ∈ {(1, 0), (-1, 0), (0, 1), (0, -1)} do
            nr = r + dr
            nc = c + dc
            if 1 ≤ nr ≤ n and 1 ≤ nc ≤ n and dist[nr][nc] == ∞ then
                dist[nr][nc] = dist[r][c] + 1
                Q.enqueue((nr, nc))
    return max(dist)
```

Soluzione meno efficiente Riportiamo anche una soluzione meno efficiente, che si basa sull'idea ingenua di confrontare ogni cella di mare con tutte le celle di terra per determinare quella più distante. Per ogni cella (r_1, c_1) che rappresenta terra, si scorre l'intera matrice alla ricerca di tutte le celle di mare (r_2, c_2) , calcolando la distanza di Manhattan tra (r_1, c_1) e (r_2, c_2) , e mantenendo il massimo tra queste. Questo approccio comporta un confronto quadratico tra coppie di celle, per una complessità complessiva pari a $\Theta(n^4)$.

```
mostDistant(boolean[][] A, int n)
```

```
    int maxSoFar = 0
    for r1 = 1 to n do
        for c1 = 1 to n do
            if A[r1][c1] then
                for r2 = 1 to n do
                    for c2 = 1 to n do
                        if not A[r2][c2] then
                            maxSoFar = max(maxSoFar, |r1 - r2| + |c1 - c2|)
    return maxSoFar
```

Presentando questa soluzione, uno studente ha scritto: "Mi spiace dell'affronto (o MonTREASON, se si vuole) che ho commesso scrivendo questo orrore".

Esercizio B1 – Livelli bonus – Punti ≥ 8

Utilizziamo un approccio di *programmazione dinamica* per calcolare il punteggio massimo accumulabile superando i livelli dall'1 all' n , nel rispetto delle regole del gioco.

Definiamo un vettore $DP[1..n]$ tale che $DP[i]$ rappresenta il massimo punteggio accumulabile partendo dal livello 1 e considerando un gioco di i livelli.

- Se c'è un solo livello, questo va superato e $DP[1] = P[1]$.
- Se ci sono due livelli, vanno superati entrambi e $DP[2] = P[1] + P[2]$.
- Per ogni livello successivo $i \geq 3$, il massimo punteggio per superarlo è dato dal valore $P[i]$ più il massimo tra:
 - il punteggio massimo accumulato superando il livello $i - 1$, e
 - il punteggio massimo accumulato superando il livello $i - 2$.

Quindi, la relazione di ricorrenza è:

$$DP[i] = P[i] + \max(DP[i - 1], DP[i - 2])$$

Alla fine, il punteggio massimo ottenibile superando il livello n sarà contenuto in $DP[n]$.

L'algoritmo esegue un singolo ciclo dal livello 3 al livello n e calcola il valore di ogni $DP[i]$ in tempo costante. La complessità temporale è dunque $\Theta(n)$. Lo spazio utilizzato è anch'esso $\Theta(n)$ a causa del vettore DP , ma può essere ridotto a $\Theta(1)$ mantenendo solo i due valori precedenti necessari al calcolo corrente.

```
int maxPoints(int [] P, int n)
{
    int DP = new int[1...n]
    DP[1] = P[1]
    DP[2] = P[1] + P[2]
    for i = 3 to n do
        DP[i] = P[i] + max(DP[i - 1], DP[i - 2])
    return DP[n]
}
```

Esercizio B2 – Massima espressività – Punti ≥ 10

Quando uno ha un bel problema, e dieci minuti prima di stampare il compito decide di modificarlo "per semplificarlo", ottiene un problema completamente diverso. In particolare, la versione originale del problema non affermava che gli interi erano tutti positivi. Avendo fatto questa modifica, esistono due soluzioni, una greedy in tempo lineare e una basata sull'algoritmo di parentesizzazione ottima visto a lezione. Ho considerato entrambe valide al 100%; per chi invece ha scritto una dimostrazione decente, ho dato un'ulteriore bonus (10%). Avendo due soluzioni, è più interessante della versione originale, ma può aver dato più da pensare e quindi era più complesso.

Soluzione basata su tecnica greedy

Soluzione basata su tecnica greedy Se tutti i valori sono strettamente positivi, conviene eseguire *prima* tutte le addizioni che stanno tra due moltiplicazioni, in modo da ingrandire gli operandi che verranno poi moltiplicati. Infatti, per $x, y, z > 0$ si ha

$$(x + y) \times z = xz + yz > x + yz,$$

quindi spostare l'addizione "dentro" la moltiplicazione non può che aumentare (o al più lasciare invariato) il valore dell'espressione. Ripetendo il ragionamento su tutte le coppie *addizione-moltiplicazione* si ottiene la strategia seguente:

1. Spezzare l'espressione in corrispondenza di ogni simbolo " $\times$ ".
2. Sommare, dentro ogni blocco così ottenuto, tutti i valori separati da "+".
3. Moltiplicare tra loro le somme dei vari blocchi.

La procedura seguente realizza l'idea con una singola scansione dell'input.

```
int maxResults(int[] V, item[] op, int n)
{
    int currentSum = V[1]
    int result = 1
    for i = 1 to n - 1 do
        if op[i] == "+" then
            currentSum = currentSum + V[i + 1]
        else
            result = result * currentSum
            currentSum = V[i + 1]
    result = result * currentSum
    return result
}
```

Dato un qualsiasi modo di parentesizzare l'espressione, si può applicare iterativamente la proprietà distributiva per spostare ogni addizione all'interno del fattore in cui compare, senza mai ridurre il valore calcolato. Alla fine di queste trasformazioni tutte le addizioni di ogni blocco delimitato da " $\times$ " vengono eseguite prima di qualsiasi moltiplicazione, e il valore ottenuto coincide con quello restituito dall'algoritmo greedy. Poiché nessuna trasformazione diminuisce il risultato, quest'ultimo è massimale.

Il ciclo principale compie un numero costante di operazioni per ciascun elemento dell'espressione: il tempo di esecuzione è quindi $\Theta(n)$.

Soluzione basata su parentesizzazione Il problema generale (anche con numeri negativi o nulli) può essere affrontato con una tecnica simile a quella vista per il prodotto di catena di matrici, sfruttando il fatto che il valore dell'espressione dipende dalla parentesizzazione, e che ogni parentesizzazione può essere vista come una combinazione di due sottoparentesizzazioni.

Sia $DP[i][j]$ il massimo valore ottenibile considerando soltanto i valori da V_i a V_j , con i corrispondenti operatori op_i fino a op_{j-1} .

- Caso base: se $i = j$, l'espressione è composta da un solo valore e non può essere ulteriormente parentesizzata:

$$DP[i][i] = V_i$$

- Passo induttivo: per ogni possibile indice k con $i \leq k < j$, dividiamo l'espressione in due sottoproblemi: la parte sinistra da i a k , e la parte destra da $k + 1$ a j . L'ultima operazione sarà op_k , e il valore complessivo sarà dato da:

$$DP[i][j] = \max_{i \leq k < j} (\text{apply}(DP[i][k], op_k, DP[k+1][j]))$$

dove la funzione `apply()` restituisce la somma o il prodotto a seconda dell'operatore.

La tabella DP viene riempita in modo bottom-up, partendo da sottoproblemi di dimensione 1 (valori singoli) e aumentando progressivamente la lunghezza delle sottostringhe.

Correttezza. L'algoritmo considera tutte le possibili parentesizzazioni in cui l'ultima operazione è effettuata tra due sottoproblemi ottimali. Grazie al principio di sottostruttura ottima, sappiamo che anche questi sottoproblemi devono essere risolti in modo ottimale per garantire il massimo valore complessivo.

Complessità. La tabella DP ha dimensione $n \times n$ e ogni cella richiede tempo $\Theta(n)$ per considerare tutte le possibili divisioni. La complessità totale è quindi $\Theta(n^3)$.

Il codice modifica la struttura del programma di parentesizzazione ottima visto a lezione:

```

int maxResults(int[] V, int[] op, int n)
{
    int[][] DP = new int[1...n][1...n]
    for  $i = 1$  to  $n$  do
         $DP[i][i] = V[i]$ 
    for  $h = 2$  to  $n$  do
        for  $i = 1$  to  $n - h + 1$  do
            int  $j = i + h - 1$ 
             $DP[i][j] = -\infty$ 
            for  $k = i$  to  $j - 1$  do
                if  $op[k] == "+"$  then
                     $DP[i][j] = \max(DP[i][j], DP[i][k] + DP[k+1][j])$ 
                else
                     $DP[i][j] = \max(DP[i][j], DP[i][k] \times DP[k+1][j])$ 
             $DP[i][j] = \max(DP[i][j], DP[i][k] \times DP[k+1][j])$ 
    return  $DP[1][n]$ 
}

```

Esercizio B3 – All paths – Punti ≥ 12

Possiamo risolvere il problema tramite backtracking. La funzione `printAllPaths()` ha il compito di inizializzare la ricerca. Essa scorre la matrice A per individuare la posizione iniziale (s_r, s_c) , ovvero la cella che contiene il valore 1, e contare quante sono le celle libere, ovvero tutte quelle diverse uguale a 0. Questo conteggio viene memorizzato nella variabile `remaining` e rappresenta il numero di celle che devono essere percorse esattamente una volta affinché il percorso sia valido.

Una volta determinato il punto di partenza, la funzione avvia la ricerca ricorsiva chiamando la procedura `apRec`, che esplora tutti i percorsi possibili dalla cella corrente (r, c) verso le quattro direzioni adiacenti, purché la cella sia valida (ovvero non sia un ostacolo, né al di fuori della matrice).

La ricorsione ha due casi base:

- Se la cella corrente è fuori dai bordi o è un ostacolo, la chiamata viene ignorata.
- Se la cella corrente è la destinazione (valore 2), si verifica se `remaining` è pari a 0 (cioè tutte le altre celle libere sono già state visitate). In tal caso, il percorso viene stampato come valido.

Per evitare di visitare due volte la stessa cella, la funzione segna temporaneamente la cella corrente come ostacolo (-1); al termine della chiamata ricorsiva, rimette il valore 0 perché sappiamo che se siamo entrati in questo if, il valore doveva essere necessariamente 0 . Ogni chiamata ricorsiva decrementa *remaining* di uno, e aggiunge la cella corrente a una struttura di tipo `STACK` per tenere traccia del percorso parziale. Al termine della ricorsione, la cella viene rimossa dalla pila e il suo valore ripristinato, consentendo così l'esplorazione di altri percorsi alternativi.

La complessità è $O(n^2 3^{n^2})$, che deriva dal fatto che in ognuna delle n^2 caselle, abbiamo 4 possibili movimenti, ma bisogna escluderne uno perché si proviene da quella direzione; mentre il fattore $O(n^2)$ deriva dall'operazione di stampa.

```
printAllPaths(int[][] A, int n)
```

```
    int remaining = 0
    int sr, sc = 0, 0
    for r = 1 to n do
        for c = 1 to n do
            if A[r][c] == 0 then
                remaining = remaining + 1
            else if A[r][c] == 1 then
                sr, sc = r, c
```

```
    STACK stack = Stack()
    apRec(A, n, sr, sc, remaining, stack)
```

```
apRec(int[][] A, int n, int r, int c, int remaining, STACK stack)
```

```
    if 1 ≤ r ≤ n and 1 ≤ c ≤ n and A[r][c] ≠ -1 then
        if A[r][c] == 2 then
            if remaining == 0 then
                S.push(⟨r, c⟩)
                print(stack)
                stack.pop()
            else
                A[r][c] = -1
                stack.push(⟨r, c⟩)
                apRec(A, n, r + 1, c, remaining - 1, stack)
                apRec(A, n, r - 1, c, remaining - 1, stack)
                apRec(A, n, r, c + 1, remaining - 1, stack)
                apRec(A, n, r, c - 1, remaining - 1, stack)
                A[r][c] = 0
                stack.pop()
```
